UniAccounts

Student Account Management System

WORK 1 — CREATIONAL DESIGN PATTERNS

Patterns implemented

Builder · Singleton · Prototype

Java project report

Documentation accompanying the Work 1 source code.

1. Introduction

This work implements the first stage of UniAccounts. The system creates student accounts, stores them in one central in-memory repository, and records account creation operations in a shared audit log. The three required patterns are Builder, Singleton, and Prototype.

2. StudentAccount model and validation

StudentAccount contains the eight fields required by the TP: id, fullName, email, phone, level, major, scholarship, and courses. The required fields are id, fullName, and email. The implementation rejects blank required values and requires the email to contain '@'. Accepted levels are L1, L2, L3, M1, and M2. Defaults are phone=null, level=L1, major=null, scholarship=false, and an empty courses list.

3. Design patterns

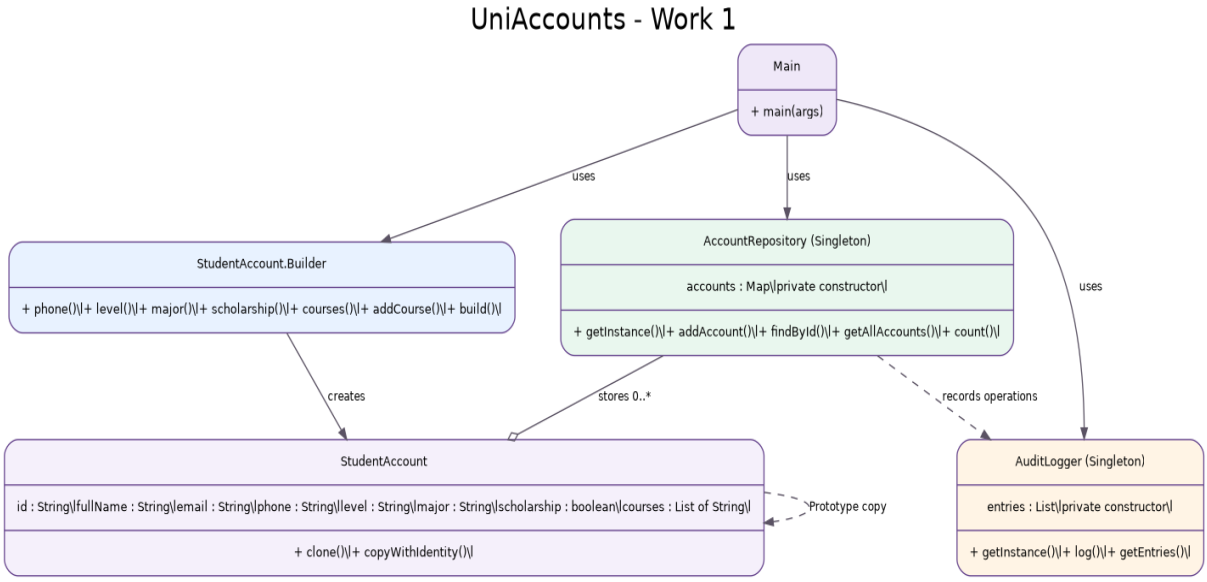
Builder — StudentAccount.Builder. The Builder creates an account step by step. Required values are supplied to the Builder constructor; optional values are configured with phone(), level(), major(), scholarship(), and addCourse(). The build() method creates the final account. This avoids a long constructor with eight arguments and prevents mistakes caused by argument order.

Singleton — AccountRepository and AuditLogger. AccountRepository uses a private constructor and a lazy, thread-safe holder to ensure one shared repository. AuditLogger uses one eagerly created instance. The repository stores accounts in a Map indexed by student id and rejects duplicate ids. This prevents different parts of the application from working with disconnected repositories or logs.

Prototype — StudentAccount. copyWithIdentity() creates a new account from a reusable academic template while replacing the id, name, and email. clone() creates an independent copy of the same account. The courses list is copied so the original and copy do not share the same mutable list. This is useful when many students share the same level, major, and courses.

4. Class design (UML)

The diagram shows the main classes and their relationships. The nested Builder creates StudentAccount objects. AccountRepository stores multiple accounts and records operations in the single AuditLogger. StudentAccount provides Prototype copying operations.



5. Execution and tests

Main creates one academic template, copies it into two student accounts using Prototype, and creates a third account directly using Builder. All three accounts are added to the repository. The program prints the stored accounts, searches for one id, attempts to add a duplicate id, checks that repeated Singleton calls return the same object, and prints the audit entries.

Expected observations: three accounts are stored; the search for id 20260002 returns the corresponding student; duplicate id 20260001 is rejected; both Singleton identity checks print true; and each successful insertion creates an audit entry.

6. Limitations and conclusion

This first version stores data in memory, so data disappears when the application exits. Email validation intentionally follows the TP's stated rule (the address must contain '@') and is not a complete email-format validator. This implementation is the foundation for Work 2.

How to run

```
mkdir -p out
javac -d out $(find src -name '*.java')
java -cp out uniaccounts.Main
```